

Guía de Referencia y Estudio: Programación Orientada a Objetos Avanzada en Java

Esta guía es un recurso docente y de consulta rápida diseñado para que los alumnos asimilen los conceptos del tema de forma progresiva, segura y práctica. Incluye explicaciones teóricas, ejemplos limpios de código, tablas comparativas y las "trampas de examen" más habituales.



1. Mapa Mental del Tema

(Estructura de conceptos y dependencias para entender el alcance global).

mindmap

root((POO Avanzada))

1. Base de Diseño

Relaciones POO

ES UN vs PUEDE HACER

TIENE UN

2. Herencia y Ambito

extends

this y super

Tabla de Visibilidad

3. Comportamiento

Sobreescritura Overriding

Sobrecarga Overloading

Anotacion Override

4. Abstraccion

Clases Abstractas

Interfaces implements

Herencia multiple

5. Polimorfismo

Enlace dinamico

Upcasting y Downcasting

instanceof

6. Organizacion y Tipos

Paquetes package

Enumerados enum



2. Ruta de Aprendizaje Recomendada

Para dominar este tema sin perderse, es fundamental abordar los conceptos en un orden acumulativo. No se puede entender el polimorfismo o las interfaces sin haber asentado perfectamente la herencia.

HERENCIA —> THIS / SUPER —> SOBRESERITURA —> POLIMORFISMO —> ABSTRACTAS / INTERFACES —> ENUMS y PAQUETES

¿Por qué este camino?

- **La Herencia** es el tronco del árbol 🌳. Establece las reglas de jerarquía y visibilidad.
- **this y super** te permiten moverte y comunicarte a través de esa jerarquía.
- **La Sobreescritura** te enseña cómo un hijo puede cambiar las normas del padre.
- **El Polimorfismo** aprovecha la sobreescritura para dar flexibilidad al código en tiempo de ejecución.
- **Las Clases Abstractas e Interfaces** son herramientas avanzadas de diseño arquitectónico que obligan a cumplir contratos basándose en todo lo anterior.



3. Idea Global: Las Relaciones de Diseño en POO

Gran parte de los errores de programación y de diseño de software ocurren por no entender qué tipo de relación une a nuestras clases. Java nos ofrece herramientas específicas para cada una:

A. Relación "ES UN" (Herencia)

Se modela utilizando la palabra clave **extends**.

- *Ejemplo:* Un Perro **es un** Animal. Un Coche **es un** Vehículo.
- *Regla:* La clase hija adquiere de forma automática los atributos y métodos de la clase padre.

B. Relación "PUEDE HACER" / "TIENE LA CAPACIDAD DE" (Interfaz)

Se modela utilizando la palabra clave **implements**.

- *Ejemplo:* Un Coche **puede hacer** la acción de Arrancable. Un Pájaro **puede hacer** la acción de Volable.
- *Regla:* La interfaz define un comportamiento común para clases que pueden no tener nada que ver entre sí.

C. Relación "TIENE UN" (Composición / Agregación)

NO se usa herencia. Se modela teniendo una instancia de otra clase como atributo.

- *Ejemplo:* Un Coche **tiene un** Motor. Un Curso **tiene un** Alumno.
- *Código:*

```
public class Coche {  
    private Motor motor; // Composición: El motor es un atributo de Coche  
}
```

⚠ **Trampa de Examen:** Un error conceptual típico es intentar heredar para relacionar elementos que en realidad son composiciones. Por ejemplo, escribir `class Motor extends Coche` es un error crítico de diseño (un Motor no es *un* Coche).

4. Herencia, Paquetes y Visibilidad

La herencia permite la reutilización de código. En Java, la herencia es **simple** (una subclase solo puede extender directamente de una única superclase).

Tabla de Visibilidad y Modificadores de Acceso

Esta tabla resume quién puede acceder a un miembro de una clase (atributo o método) según su modificador:

Modificador	Dentro de la misma Clase	En el mismo Paquete	Subclases en otro Paquete	En cualquier otro lugar
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
default (sin especificar)	Sí	Sí	No	No
private	Sí	No	No	No

El uso de **this** y **super**

- **this:** Hace referencia al objeto de la clase actual. Se usa habitualmente en constructores para romper la ambigüedad entre parámetros y atributos.
- **super:** Hace referencia a la parte del objeto que corresponde a la superclase (el padre).
- **super(...):** Invoca al constructor del padre.

```
// Archivo: Persona.java en paquete 'personas'
package personas;

public class Persona {
    protected String nombre; // protected: visible en subclases de
    otros paquetes

    public Persona(String nombre) {
        this.nombre = nombre; // 'this' resuelve la ambigüedad con el
        parámetro
    }
}

// Archivo: Estudiante.java en paquete 'estudiantes'
package estudiantes;
import personas.Persona;

public class Estudiante extends Persona {
    private String matricula;

    public Estudiante(String nombre, String matricula) {
        super(nombre); // REGLA: super() tiene que ser
        obligatoriamente la PRIMERA instrucción
        this.matricula = matricula;
    }
}
```

⚠ Dificultades y Trampas en Exámenes:

- **El Constructor por defecto Fantasma:** Si el padre tiene un constructor personalizado con parámetros (como `Persona(String nombre)`) y *no* tiene un constructor vacío, la subclase está **obligada** a llamar explícitamente a `super(nombre)` en la primera línea de su constructor. Si no lo hace, el código no compilará porque Java intentará buscar un constructor vacío en el padre (`super()`) que no existe.
- **Acceso a miembros private:** Aunque una subclase hereda todos los miembros del padre, **no puede acceder directamente** a aquellos que sean `private`. Debe interactuar con ellos mediante métodos públicos o protegidos (getters/setters).

5. Sobrecarga (Overloading) vs. Sobreescritura (Overriding)

Es muy común confundir estos dos términos debido a la similitud de sus nombres. Esta tabla marca la diferencia insalvable entre ambos:

Característica	Sobrecarga (Overloading)	Sobreescritura (Overriding)
Ubicación	Generalmente en la misma clase.	Siempre en clases distintas con relación de herencia.
Nombre	Idéntico.	Idéntico.
Parámetros (Firma)	Obligatoriamente diferentes (en tipo, cantidad u orden).	Exactamente iguales.
Retorno	Puede cambiar libremente.	Debe ser idéntico o un subtipo compatible.
Ámbito de Acceso	Puede ser diferente.	No puede ser más restrictivo que el del padre.
Anotación	No lleva.	Se utiliza @Override (muy recomendado).

Ejemplo Práctico

```
public class Calculadora {
    // SOBRECARGA: Mismo nombre, firmas de parámetros distintas
    public int sumar(int a, int b) { return a + b; }
    public double sumar(double a, double b) { return a + b; }
}

public class CalculadoraCientifica extends Calculadora {
    // SOBREESCRITURA: Cambia el comportamiento de un método heredado
    con la misma firma
    @Override
    public int sumar(int a, int b) {
        System.out.println("Realizando suma de alta precisión...");
        return super.sumar(a, b);
    }
}
```

 **Dificultades y Trampas en Exámenes:**

- **La trampa del retorno en la sobrecarga:** Cambiar únicamente el tipo de retorno **no** sobrecarga un método y generará un error de compilación. Las firmas de parámetros *deben* cambiar.

```
public int calcular() { ... }
public double calcular() { ... } // ❌ ERROR DE COMPILACIÓN: Firmas idénticas
```
- **La anotación @Override:** Aunque es opcional, si no la usas y cometes un error tipográfico en el nombre del método (ej. escribir *summar* en vez de *sumar*), Java asumirá que es un método nuevo (sobrecarga) en lugar de una sobreescritura, lo que provocará fallos lógicos difíciles de depurar en el polimorfismo.

6. Polimorfismo y Enlace Dinámico

El polimorfismo es la capacidad de almacenar un objeto de una subclase en una variable cuyo tipo de referencia es de la superclase.

Enlace Dinámico

Cuando llamamos a un método sobreescrito a través de una referencia genérica, Java no decide qué método ejecutar según el tipo de la variable (referencia), sino según el **tipo del objeto real** en memoria durante el tiempo de ejecución.

// Tipo de referencia: Persona | Objeto real: Estudiante

```
Persona p = new Estudiante("Carlos", "A0112");
```

`p.saludar();` // Se ejecutará el método 'saludar()' de Estudiante, no el de Persona.

Conversión de Tipos: Upcasting y Downcasting

- **Upcasting (Automático e intrínsecamente seguro):** Asignar un objeto hijo a una referencia padre.

```
Persona p = new Estudiante("Luis", "B23");
```
- **Downcasting (Manual e inseguro):** Recuperar el tipo real del objeto para acceder a sus métodos específicos. Requiere una comprobación previa con **instanceof** para evitar fallar en producción.

// Supongamos que Estudiante tiene el método exclusivo

'entregarPractica()' que Persona no tiene.

```
if (p instanceof Estudiante) {
```

```
    Estudiante est = (Estudiante) p; // Downcasting explícito
```

```
    est.entregarPractica();           // Ahora sí podemos acceder al
```

```
    método específico
```

```
}
```

⚠ Dificultades y Trampas en Exámenes:

- **La trampa del acceso limitado:** Con una variable de tipo `Persona` `p`, **solo** se pueden invocar los métodos declarados en la clase `Persona`. Intentar hacer `p.entregarPractica()` directamente dará un **error de compilación**, aunque sepamos que la variable contiene un estudiante en ese instante.
- **El crash `ClassCastException`:** Si intentas forzar un casting de un objeto a una clase que no le corresponde (por ejemplo, convertir una `Persona` que en realidad es un `Profesor` a un `Estudiante`), el programa compilará pero se romperá inmediatamente al ejecutarse. Usa siempre `instanceof`.



7. Clases Abstractas vs. Interfaces

Ambas estructuras impiden crear objetos directos mediante `new` y sirven para definir plantillas obligatorias para otras clases, pero responden a necesidades arquitectónicas distintas:

Característica	Clase Abstracta (abstract)	Interfaz (interface)
Relación principal	"ES UN" (Estructura jerárquica).	"PUEDA HACER" (Habilidades comunes).
Herencia	Una clase solo hereda de una abstracta.	Una clase puede implementar múltiples interfaces.
Atributos	Puede tener variables de instancia normales.	Solo constantes (<code>public static final</code>).
Métodos	Puede mezclar métodos normales (con código) y métodos abstractos (sin cuerpo).	Por defecto todos son abstractos (sin cuerpo), salvo métodos <code>default</code> o <code>static</code> .
Constructores	Sí tiene (para inicializar variables heredadas).	No tiene bajo ningún concepto.

```
public interface Volar {  
    void volar(); // Implícitamente public abstract  
}
```

```
public abstract class Ave {
```

```

        protected String especie;
        public Ave(String especie) { this.especie = especie; }

        public abstract void emitirCanto(); // Método que cada ave
        concreta debe definir
    }

    // Subclase que hereda una estructura e implementa una habilidad
    voluntaria
    public class Halcon extends Ave implements Volable {
        public Halcon() { super("Accipitradae"); }

        @Override
        public void emitirCanto() { System.out.println(";Chillido
        agudo!"); }

        @Override
        public void volar() { System.out.println("Vuelo en picado a alta
        velocidad"); }
    }

```

8. Elementos Organizativos y Tipos Especiales

Paquetes (package e import)

Los paquetes organizan los archivos de código de forma jerárquica en carpetas de disco.

- **Regla física:** El nombre del paquete declarado en la primera línea de código debe coincidir estrictamente con la estructura de directorios del disco.
- **Regla de importación:** Para usar una clase de otro paquete debes importarla con `import nombrepaquete.Clase;` o usar su nombre cualificado completo.

Enumerados (enum)

Un tipo de clase especial que limita los valores válidos de una variable a una lista cerrada de opciones predefinidas en mayúsculas.

```

public enum NivelAcceso {
    INVITADO, USUARIO, ADMINISTRADOR
}

```

- **values():** Método automático que retorna un array con todos los elementos definidos en el enum (muy útil para iterar en bucles).

- **valueOf(String nombre):** Traduce un String al tipo enum correspondiente.

⚠ Dificultades y Trampas en Exámenes:

- **Error por mayúsculas/minúsculas en valueOf():** Es extremadamente estricto. Si ejecutas `NivelAcceso.valueOf("invitado")` en minúsculas, lanzará inmediatamente una excepción `IllegalArgumentException` y detendrá el programa.
 - *Solución segura:* `NivelAcceso nivel = NivelAcceso.valueOf(entradaTeclado.trim().toUpperCase());`



9. Proyecto de Demostración Integrador

Este es un único código cohesivo pensado para que lo copies, lo compiles y juegues con él.
Implementa e ilustra casi el 100% de los conceptos avanzados del tema de golpe:

```
// =====
// 1. DEFINICIÓN DE INTERFACES Y ENUMS
// =====
enum TipoCombustible {
    GASOLINA, ELECTRICO, HIBRIDO
}

interface Conducible {
    void conducir(); // Capacidad: "Puede ser conducido"
}

// =====
// 2. CLASE ABSTRACTA (PLANTILLA BASE)
// =====
abstract class Vehiculo {
    protected String marca;
    protected TipoCombustible combustible; // Uso de enumerado

    // Constructor que será invocado por las subclases
    public Vehiculo(String marca, TipoCombustible combustible) {
        this.marca = marca;
        this.combustible = combustible;
    }

    // Método normal (comportamiento heredado ya implementado)
    public void mostrarInfo() {
        System.out.println("Vehículo " + marca + " que utiliza: " +
            combustible);
    }
}
```

```

    }

    // Método abstracto (cada hijo debe programar obligatoriamente su
lógica)
    public abstract void encender();
}

// =====
// 3. SUBCLASES CONCRETAS (HERENCIA Y CONTRATOS)
// =====
class Coche extends Vehiculo implements Conducible {
    private int puertas;

    public Coche(String marca, TipoCombustible combustible, int
puertas) {
        super(marca, combustible); // Llamada obligatoria al
constructor padre
        this.puertas = puertas;
    }

    @Override
    public void encender() {
        System.out.println("El coche " + marca + " enciende su motor
silenciosamente.");
    }

    @Override
    public void conducir() {
        System.out.println("Conduciendo el coche de " + puertas + "
puertas por la carretera...");
    }

    // Método exclusivo de Coche (no está en Vehiculo)
    public void abrirMaletero() {
        System.out.println("Maletero del coche abierto.");
    }
}

class Moto extends Vehiculo implements Conducible {

    public Moto(String marca, TipoCombustible combustible) {
        super(marca, combustible);
    }
}

```

```

        @Override
        public void encender() {
            System.out.println("La moto " + marca + " ruge al
arrancar.");
        }

        @Override
        public void conducir() {
            System.out.println("Conduciendo la moto tomando curvas
pronunciadas...");
        }
    }

// =====
// 4. CLASE PRINCIPAL CON DEMOSTRACIÓN DE POLIMORFISMO
// =====
public class AppPOO {
    public static void main(String[] args) {
        // POLIMORFISMO: Creamos una lista de tipo genérico Vehiculo
        Vehiculo[] flota = new Vehiculo[2];
        flota[0] = new Coche("Toyota", TipoCombustible.HIBRIDO, 5);
// Upcasting automático
        flota[1] = new Moto("Yamaha", TipoCombustible.GASOLINA);    //
Upcasting automático

        System.out.println("--- INICIO DE DEMOSTRACIÓN ---");
        for (Vehiculo v : flota) {
            v.mostrarInfo();

            // ENLACE DINÁMICO: Java sabe en ejecución a qué método
llamar
            v.encender();

            // USO DE INSTANCEOF Y DOWNCASTING SEGURO
            if (v instanceof Coche) {
                Coche c = (Coche) v; // Conversión manual segura
                c.abrirMaletero();    // Acceso al método exclusivo
            }

            System.out.println("-----");
        }
    }
}

```

